Control Systems-II

" Stabilizing an Inverted Pendulum Using LQRTechnique"

Submitted by:

- MUHAMMAD BILAL (2022-MC-41)
- SARAM ABDULLAH (2022-MC-49)
- FAIQ AHMED ZIA (2022-MC-59)

Submitted to:

Dr. Mohsin Rizwan

DEPARTMENT OF MECHATRONICS AND CONTROL ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY LAHORE

Introduction

The inverted pendulum is a fundamental system in control engineering that consists of a pendulum mounted on a moving cart. It is inherently unstable, as any small deviation from the upright position causes it to fall unless corrective actions are taken. This project focuses on designing and implementing a control system to stabilize the inverted pendulum using the Linear Quadratic Regulator (LQR) technique

Understanding the Linear Quadratic Regulator (LQR) Technique:

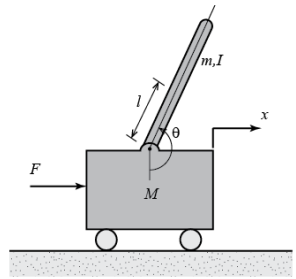
The Linear Quadratic Regulator (LQR) is a modern control technique used to design controllers that achieve the best performance by minimizing a specific cost function. The cost function typically balances the trade-off between the system's performance (like how quickly and accurately it reaches the desired state) and the control effort (how much energy or power the control system uses). In the context of stabilizing an inverted pendulum, LQR helps determine the optimal control input to keep the pendulum balanced at its upright position by continuously adjusting the cart's position.

We use LQR because it provides a systematic way to handle complex systems with multiple states (like position, velocity, and angle), ensuring both stability and efficient performance. It is particularly significant because it can handle real-world constraints and disturbances, making it ideal for challenging tasks like balancing an inverted pendulum. make a suitable heading for it

Problem Definition

The inverted pendulum system involves a cart that moves linearly with a pendulum mounted on top. The main challenge is to stabilize the pendulum in an upright position while controlling the cart's movement within predefined limits, requiring precise control strategies to manage the system's inherent instability.

System Dynamics



Mathematical Modeling

State-Space

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\phi} \\ \ddot{\phi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-(I+ml^2)b}{I(M+m)+Mml^2} & \frac{m^2gl^2}{I(M+m)+Mml^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-mlb}{I(M+m)+Mml^2} & \frac{mgl(M+m)}{I(M+m)+Mml^2} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \phi \\ \dot{\phi} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{I+ml^2}{I(M+m)+Mml^2} \\ 0 \\ \frac{ml}{I(M+m)+Mml^2} \end{bmatrix} u$$

$$\mathbf{y} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \phi \\ \dot{\phi} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u$$

Parameters	Description	Values
m	Mass of the pendulum	0.5 kg
(M)	Mass of the cart	1.3 kg
g	Gravitational force	9.8 ms^-2
d	damping	3
L	Length of the pendulum	0.41m
l	Length of track	1.2573 m

MATLAB CODE FOR STATE SPACE

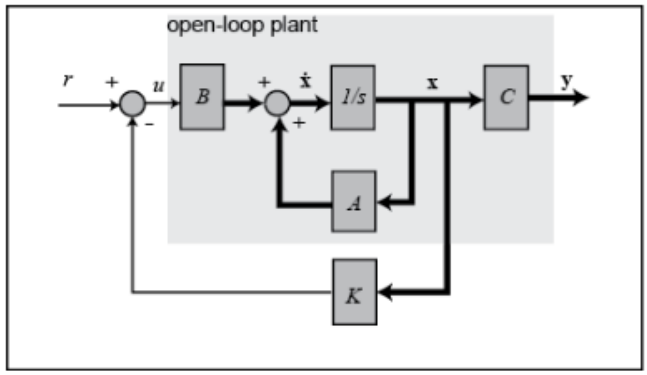
```
M = 1.3;
m = 0.5;
b = 0.1;
I = 0.1121;
g = 9.8;
l = 0.41;

p = I*(M+m)+M*m*l^2; %denominator for the A and B matrices

A = [0      1            0          0;
    0 -(I+m*l^2)*b/p  (m^2*g*l^2)/p   0;
    0      0            0          1;
    0 -(m*l*b)/p       m*g*l*(M+m)/p  0];
B = [     0;
    (I+m*l^2)/p;
         0;
       m*l/p];
C = [1 0 0 0;
    0 0 1 0];
D = [0;
    0];

states = {'x' 'x_dot' 'phi' 'phi_dot'};
inputs = {'u'};
outputs = {'x'; 'phi'};

sys_ss = ss(A,B,C,D,'statename',states,'inputname',inputs,'outputname',outputs)
```

RESULT

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -0.07017 & 2.579 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -0.1284 & 22.64 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0.7017 \\ 0 \\ 1.284 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

OPEN LOOP POLES



$$\lambda_1 = 0$$

$$\lambda_2 = -0.0556$$

$$\lambda_3 = -3.4135$$

$$\lambda_4 = +3.4060$$

As you can see, there is one right-half plane pole at 3.4060. This should confirm my intuition that the system is unstable in open loop.

Linear Quadratic Regulator (LQR) Design

LQR Formulation

The LQR minimizes the cost function:

J = ∫ (X^T Q X + U^T R U) dt

Where Q is the state weighting matrix, R is the control effort weighting matrix, and U is the control input.

The optimal control law is:

U = -KX,

where K is the gain matrix calculated by solving the Riccati equation.

Using MATLAB, the gain matrix K is computed as follows:

```
Q =

     5000        0        0        0

        0        0        0        0

        0        0      100        0
   0        0        0        0
```

```
R = 1;
K = lqr(A, B, Q, R);
disp('LQR Gain Matrix K:');
disp(K);
```

Parameter Selection

Simulation values were chosen as follows:

Q = diag([5000, 0, 100, 0]);

R = 1;

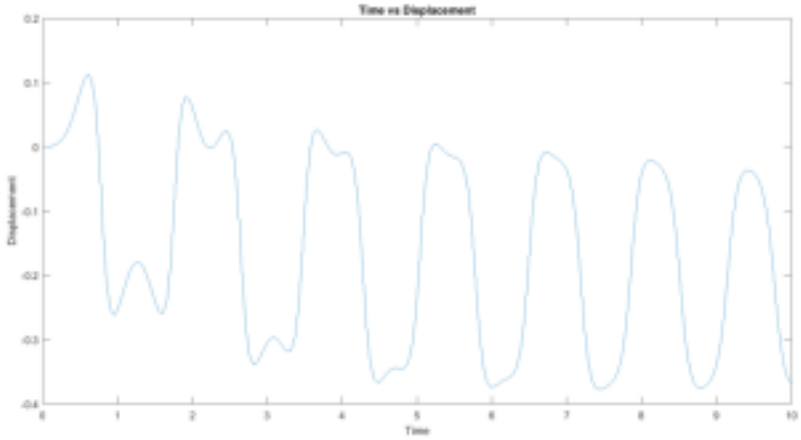
$$K = \begin{bmatrix} -70.7107 & -48.1178 & 177.3055 & 40.9380 \end{bmatrix}$$

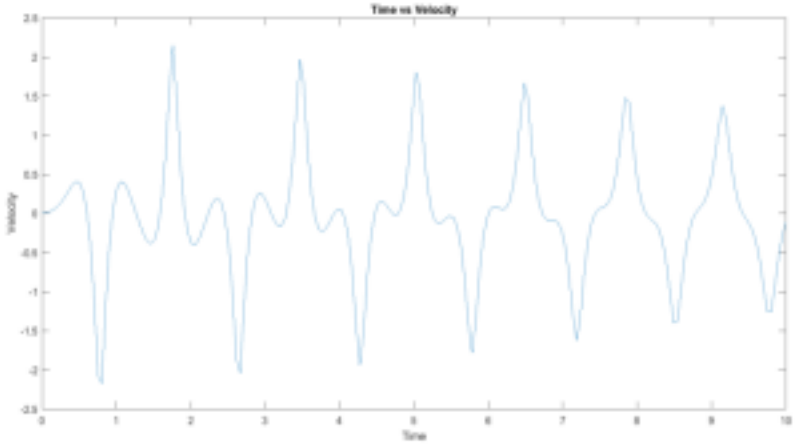
Mathematical modelling using Matlab

```
clear all, close all, clc
m = .5;
M = 1.3;
L = 0.41;
g = -10;
d = 1;
tspan = linspace(0, 10, 200);
%tspan = 0:0.1:10;
y0 = [0; 0; pi; .5];
[t,y] = ode45(@(t,y)cartpend(y,m,M,L,g,d,0),tspan,y0);
%subplot (2, 1, 1)
displacement=y(:,1);
plot(t,displacement)
title('Time vs Displacement')
xlabel('Time')
ylabel('Displacement')
figure;
```

```
%subplot (2, 2, 1)
velocity=y(:,2);
plot(t,velocity)
title('Time vs Velocity')
xlabel('Time')
ylabel('Velocity')
%Run Animation
figure(1); % Create a single figure window
set(gcf, 'Color', 'k'); % Set figure background to black
for k = 1:length(t)
 drawcartpend_bw(y(k, :), m, M, L);
 pause(0.01); % Pause to match simulation time step
end
```

RESULTS





Code Implementation on Ardiuno

```
/* ============================================================
   INVERTED PENDULUM - SMOOTH LQR + CART CENTERING + NO VIBRATION
   Arduino Mega 2560
   ============================================================ */

#include <Arduino.h>

// ===================== LQR GAINS =====================
float K1 = 0.00015;   // cart position gain (centering)
float K2 = 0.008;     // cart velocity gain
float K3 = 18.0;       // pendulum angle gain (reduced for
smoothness)
float K4 = 0.6;       // angular velocity gain (reduced)

// PWM limits
const int MAX_PWM = 80;
const float PWM_SCALE = 0.7;  // Soft scaling to reduce jerks

const int BALANCE_WINDOW = 80;   // Only try to balance within ±80
counts
const int UPRIGHT_DEADZONE = 2;  // Ignore tiny deviations to stop
micro-corrections

// ===================== PINS =====================
const int cartEncA = 2;
const int cartEncB = 3;
const int pendEncA = 18;
const int pendEncB = 19;

const int R_EN  = 7;
const int L_EN  = 8;
const int R_PWM = 9;
const int L_PWM = 10;

// ===================== VARIABLES =====================
volatile long pendCounts = 0;
volatile long cartCounts = 0;

long lastPend = 0;
long lastCart = 0;
```

```
float theta_dot_filtered = 0.0;
float x_dot_filtered = 0.0;

unsigned long lastLoopTime = 0;

// Low-pass filter coefficient (0 < alpha < 1)
const float alpha = 0.85;

// ===================== SETUP =====================
void setup() {
  Serial.begin(115200);

  pinMode(R_EN, OUTPUT); pinMode(L_EN, OUTPUT);
  digitalWrite(R_EN, HIGH); digitalWrite(L_EN, HIGH);
  pinMode(R_PWM, OUTPUT); pinMode(L_PWM, OUTPUT);

  pinMode(cartEncA, INPUT); pinMode(cartEncB, INPUT);
  pinMode(pendEncA, INPUT); pinMode(pendEncB, INPUT);

  attachInterrupt(digitalPinToInterrupt(cartEncA), ISR_cart,
RISING);
  attachInterrupt(digitalPinToInterrupt(pendEncA), ISR_pendulum,
RISING);

  noInterrupts();
  pendCounts = 0;
  cartCounts = 0;
  interrupts();

  lastLoopTime = millis();
  lastPend = pendCounts;
  lastCart = cartCounts;

  Serial.println("System ready: Smooth LQR no vibration.");
  delay(3000);
}

// ===================== MAIN LOOP =====================
void loop() {
  unsigned long now = millis();
```

```
  if (now - lastLoopTime < 2) return; // ~500 Hz
  float dt = (now - lastLoopTime)/1000.0;
  lastLoopTime = now;

  // ===== Read encoders safely =====
  noInterrupts();
  long currentPend = pendCounts;
  long currentCart = cartCounts;
  interrupts();

  // ===== Normalize pendulum angle =====
  long normalizedPend = currentPend % 2048;
  if (normalizedPend < 0) normalizedPend += 2048;

  // ===== Compute velocities =====
  float x_dot_raw = (currentCart - lastCart)/dt;
  float theta_dot_raw = -(normalizedPend - lastPend)/dt; // upright
positive

  // ===== Low-pass filter =====
  x_dot_filtered = alpha * x_dot_filtered + (1-alpha) * x_dot_raw;
  theta_dot_filtered = alpha * theta_dot_filtered + (1-alpha) *
theta_dot_raw;

  lastPend = normalizedPend;
  lastCart = currentCart;

  // ===== State variables =====
  float x = currentCart;
  float theta = 1024.0 - normalizedPend;

  // Only balance within window
  if (abs(theta) < BALANCE_WINDOW) {
    // LQR control
    float u = -(K1*x + K2*x_dot_filtered + K3*theta +
K4*theta_dot_filtered);

    // Deadzone to avoid micro jitter at upright
    if (abs(theta) < UPRIGHT_DEADZONE) u = 0;

    driveMotor(u * PWM_SCALE);
```

```
  } else {
    driveMotor(0); // Outside balance window
  }
}

// ===================== MOTOR DRIVER =====================
void driveMotor(float pwm) {
  int output = constrain((int)pwm, -MAX_PWM, MAX_PWM);

  if (output > 0) { analogWrite(R_PWM, output); analogWrite(L_PWM,
0); }
  else if (output < 0) { analogWrite(R_PWM, 0); analogWrite(L_PWM,
abs(output)); }
  else { analogWrite(R_PWM, 0); analogWrite(L_PWM, 0); }
}

// ===================== INTERRUPTS =====================
void ISR_cart() {
  if (digitalRead(cartEncB) == HIGH) cartCounts++;
  else cartCounts--;
}

void ISR_pendulum() {
  if (digitalRead(pendEncB) == HIGH) pendCounts++;
  else pendCounts--;
}
```